



<http://algs4.cs.princeton.edu>

3.2 BINARY SEARCH TREES

- ▶ *BSTs*
- ▶ *ordered operations*
- ▶ *deletion*



<http://algs4.cs.princeton.edu>

3.2 BINARY SEARCH TREES

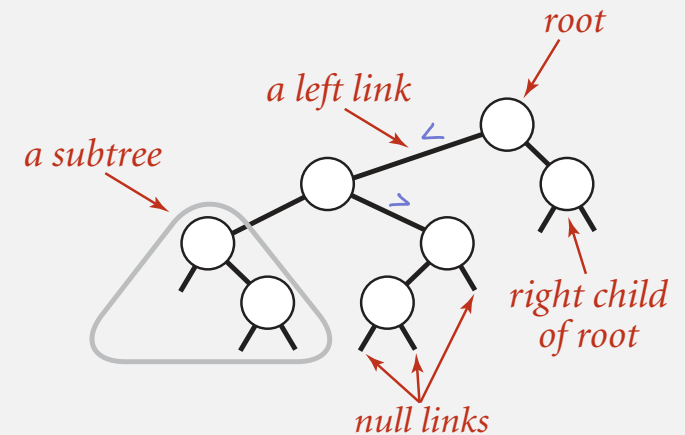
- ▶ *BSTs*
- ▶ *ordered operations*
- ▶ *deletion*

اهفر منه،
واليمين اكبر
منه او يساوية

• **Definition.** A BST is a **binary tree** in **symmetric order**.

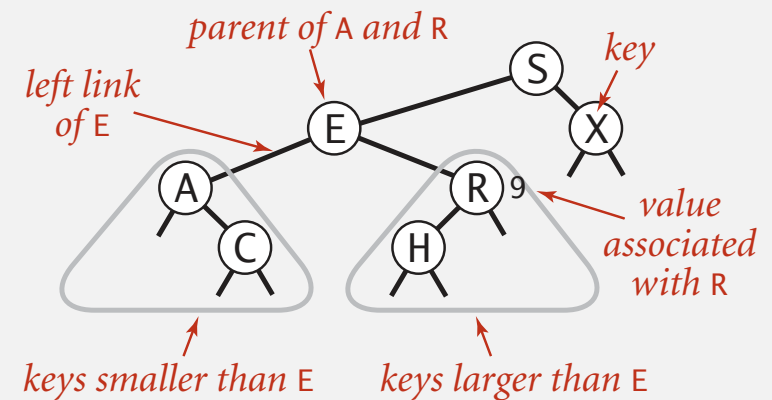
A binary tree is either:

- Empty.
- Two disjoint binary trees (left and right).



Symmetric order. Each node has a key, and every node's key is:

- Larger than all keys in its left subtree.
- Smaller than all keys in its right subtree.

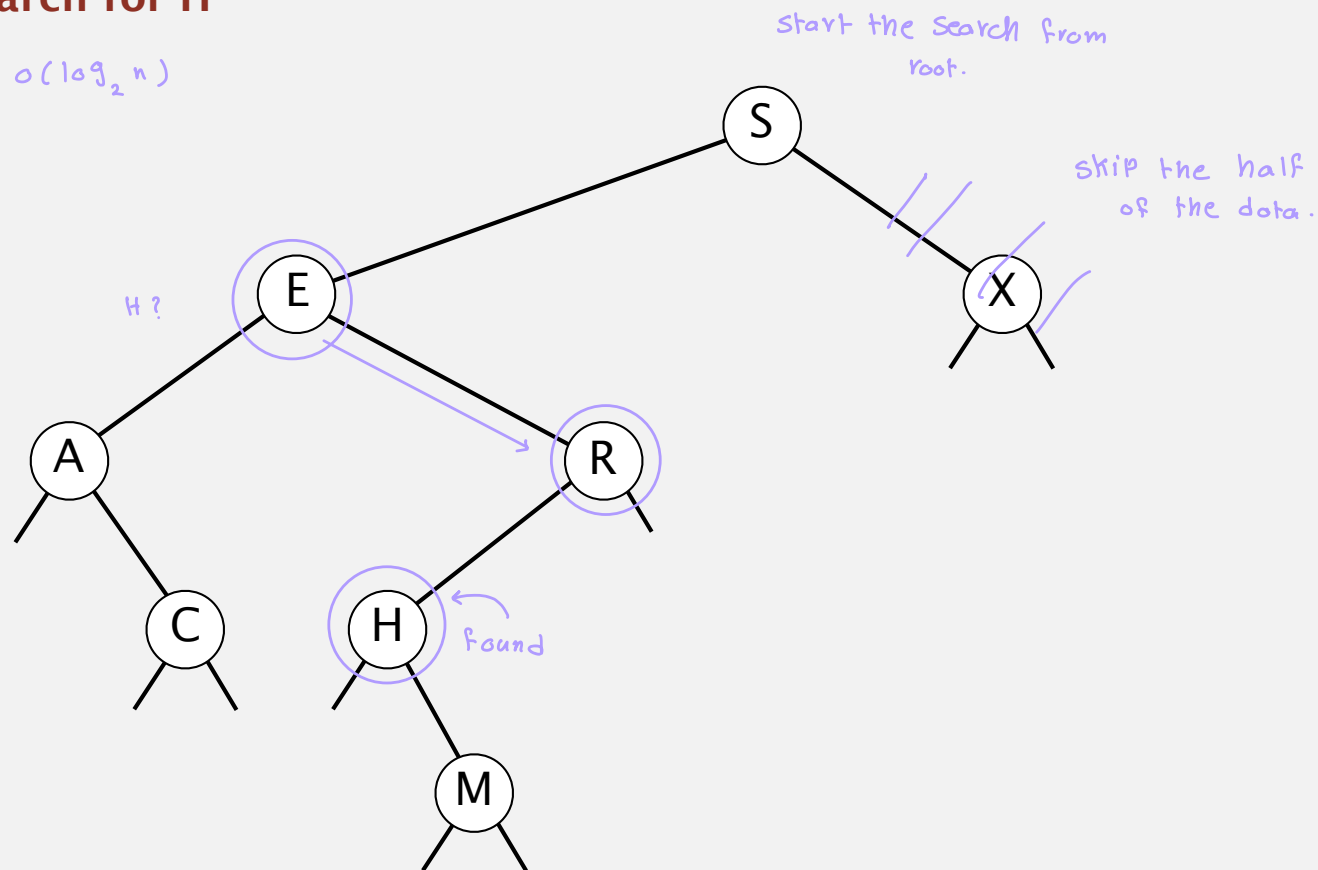


Binary search tree demo

Search. If less, go left; if greater, go right; if equal, search hit.

successful search for H

$n = 8$
best = 4 $\rightarrow O(\log_2 n)$

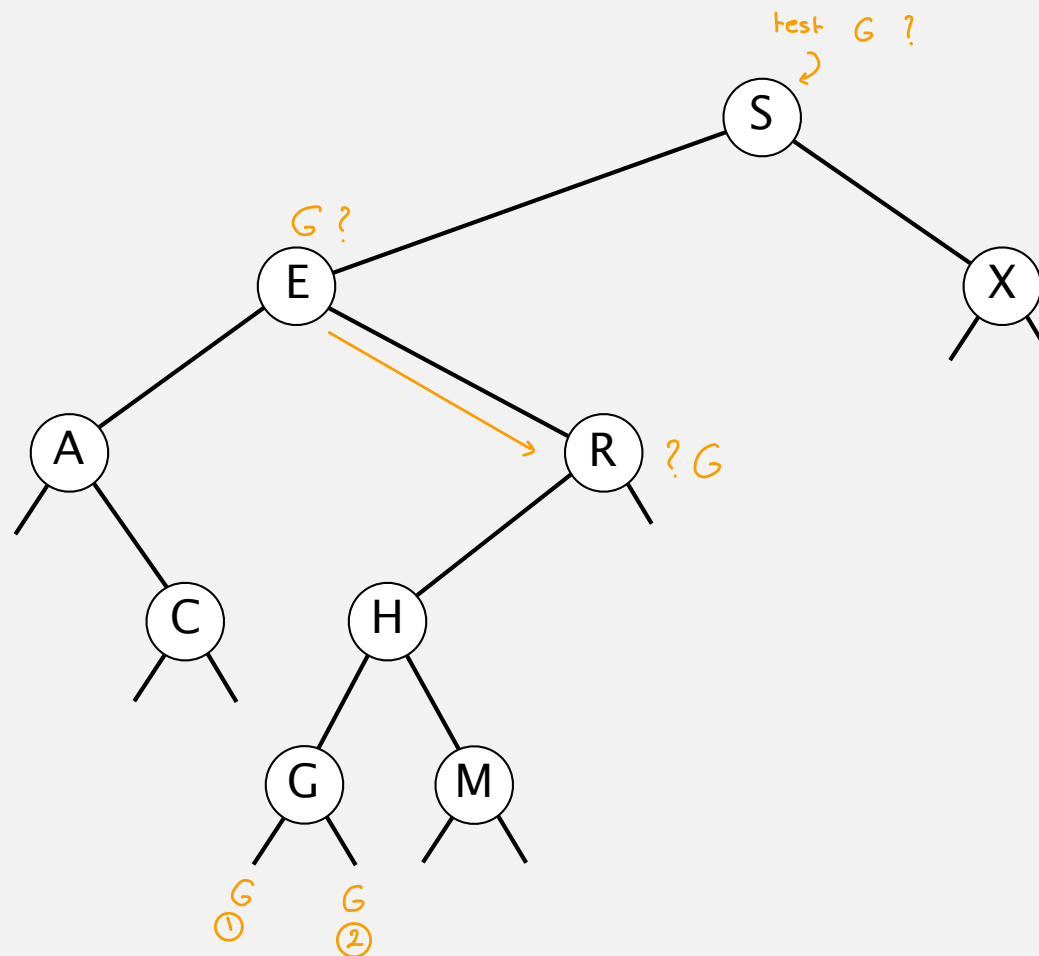


Binary search tree demo

Insert. If less, go left; if greater, go right; if null, insert.

insert G

1-
2- $L \leq N \leq R$
 $L \leq N < R$



BST representation in Java

Java definition. A BST is a reference to a root Node.

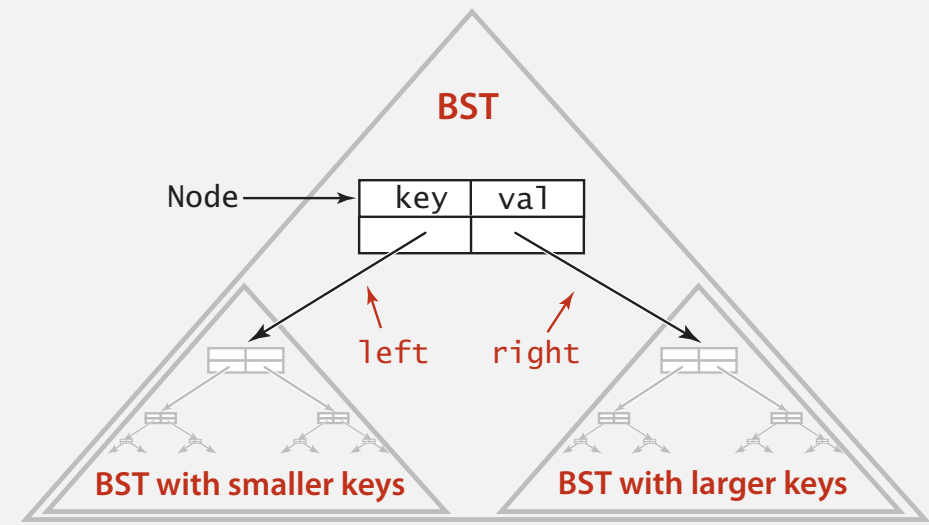
A Node is composed of four fields:

- A Key and a Value.
- A reference to the left and right subtree.

↑ smaller keys ↑ larger keys

Key : ID	

```
private class Node
{
    private Key key; must be unique data.
    private Value val;
    private Node left, right;
    public Node(Key key, Value val)
    {
        this.key = key;
        this.val = val;
    }
}
```



Key and Value are generic types; Key is Comparable

BST implementation (skeleton)

```
public class BST<Key extends Comparable<Key>, Value>
{
```

```
    private Node root;
```

← root of BST

```
    private class Node
    { /* see previous slide */ }
```

```
    public void insertput(Key key, Value val)
    { /* see next slides */ }
```

```
    public Value searchget(Key key)
    { /* see next slides */ }
```

```
    public void delete(Key key)
    { /* see next slides */ }
```

```
    public Iterable<Key> iterator()
    { /* see next slides */ }
```

```
}
```

BST search: Java implementation

↪ must be order

Get. Return value corresponding to given key, or null if no such key.

Tree Search:

عندنا ثلاث احتمالات

1. يا إما تساويها .
2. يا إما اكبر منها .
3. يا إما انها null .
4. يا إما اقل منها .

إذا كانت (External) :

— معناة : null مثل موجودة
ونعمل return لل Value .

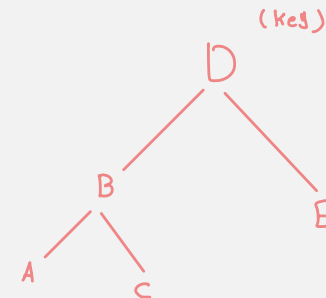
— وإذا كانت اقل بتنادي

ال Function على الجهة اليسار .

— إذا كانت تساوي معناة
لنعمل بارجع ال Value .

— إذا كانت اكبر بتنادي
ال Function على الجهة اليمين .

```
public Value get(Key key)
{
    Node x = root;
    while (x != null)
    {
        int cmp = key.compareTo(x.key);
        if (cmp < 0) x = x.left;
        else if (cmp > 0) x = x.right;
        else if (cmp == 0) return x.val;
    }
    return null; Element does not exist.
}
```



$O(\log n + 1)$ → we add one to search

Cost. Number of compares is equal to 1 + depth of node.

* كل ما كتبت ال array
هاد اسرع في الحسابات

1	3	5	7	9
0	1	2	3	4

Search for (7)

```
first = 0
last = 4
mid = (0+4) / 2 = 2
a[2] = 7
5 < 7
```

first = mid + 1 = 3
last = 4
mid = (3 + 4) / 2 = 3

$a[3] = 7$ Stop
7 = 7 found

Search for (6):

```
first = 0
last = 4
mid = (0+4) / 2 = 2
```

$a[2] : 6$
 $5 < 6$

```
first = mid + 1 = 3  
last = 4  
mid = (3 + 4) / 2 = 3
```

$$\frac{a[3] \cdot 6}{7 \cdot 6}$$

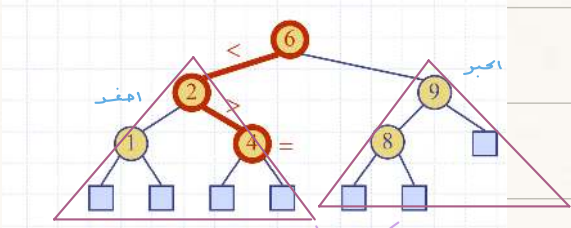
لهم انهم يعرفون مني اهل
بييوتك الـ last .

first = 3
lost = mid - 1 = 2

لكن الـ last افضل من الـ first
 first دائم الـ first افضل من last
 لان الـ first يجي من قدام ، والـ last
 من واء ، فالـ first قيمه كـ index افضل من الـ last .

فالمن بصركذا الأول أكبر من الأول معناه يعمل (Stop) و not found .

$left > root \leq right$
 $\star key(u) \leq key(v) \leq key(w)$



→ immediate successor: left وبعدين لاقص right بنحرك وحدة

☆ immediate Predecessor: $\rightarrow$ الي قبله مباشرة ، بتحول وحده left لبعدين لافق right.

ایشان اگر قیمه فیما بیكون اكثر (ایشان اگر قیمه فیما بیكون اكثر شي، right.

مين قبل الـ ١٩٤٥ ؟ : يتحرك وحدة ١٩٤٥ بعدين

صين الى بعد ك ؟ : يتحرك وحده right .

right , right , right لاقص الـ right وتكون حذى قبل الـ كـ .

left weq̄x, left, left, left

ويكون هذا
الي بعد ك.

BST insert

Put. Associate value with key.

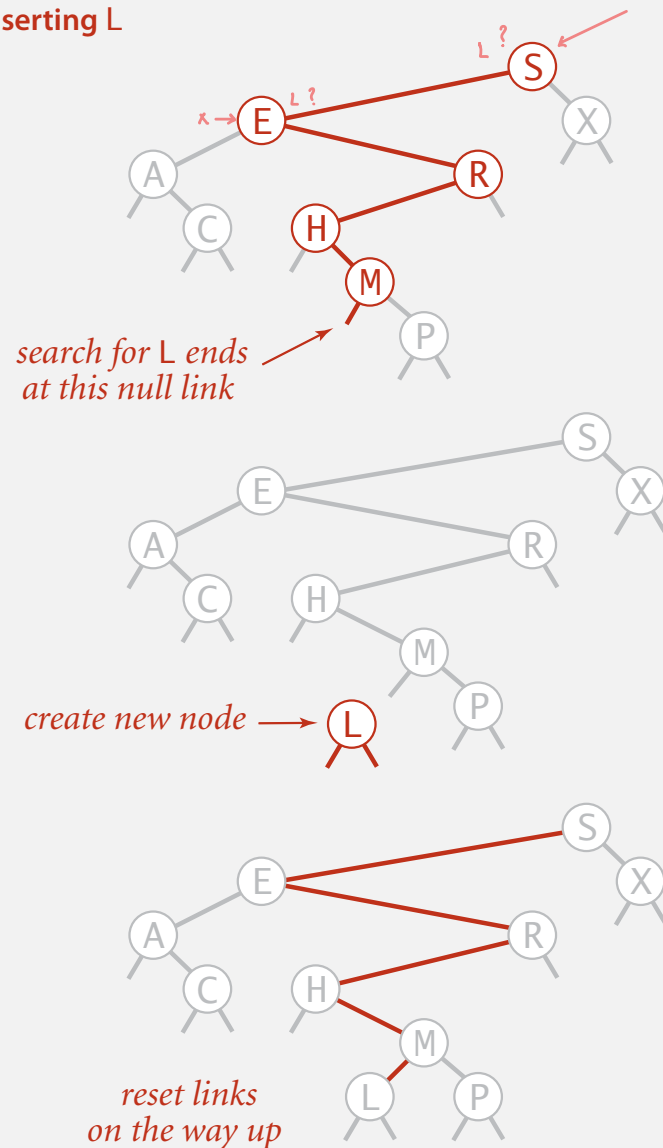
Search for key, then two cases:

- Key in tree $\Rightarrow$ reset value.
- Key not in tree $\Rightarrow$ add new node.

✳️ اول ما اخيف رقم يكون هو ال root .
 بعدين بضيف رقم ثاني بيقل منه بال root
 اذا اوفر بيزوج left واذا اكبر بيزوج right وهكذا .
 ✳️ البحث يبدأ من ال root يقارن الي يحصل الرقم
 اذا هو موجود يعبر (Not found) .

Inorder tree: $L < \text{root} < R$

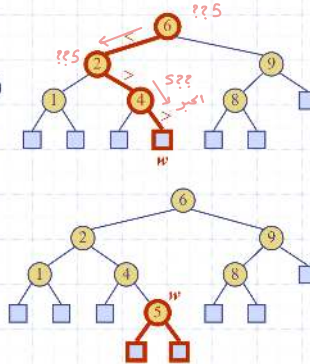
inserting L



Insertion into a BST

Insertion

- ◆ To perform operation **put**(k, o), we search for key k (using TreeSearch)
- ◆ Assume k is not already in the tree, and let w be the leaf reached by the search
- ◆ We insert k at node w and expand w into an internal node
- ◆ Example: insert 5



20

Insertion

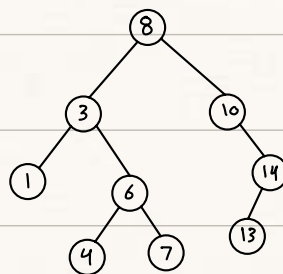
- ◆ Insert the following elements in a BST



© 2014 Goodrich, Tamassia, Goldwasser

Binary Search Trees

22



BST insert: Java implementation

Put. Associate value with key.

```
public void put(Key key, Value val)
{ root = put(root, key, val); }

private Node put(Node root x, Key key, Value val)
{
    if (x == null) return new Node(key, val);
    int cmp = key.compareTo(x.key);
    if (cmp < 0)
        x.left = put(x.left, key, val);
    else if (cmp > 0)
        x.right = put(x.right, key, val);
    else if (cmp == 0)
        x.val = val;
    return x;
}
```

Found, 0 → (points to cmp == 0)

right (points to x.right)

concise, but tricky,
recursive code;
read carefully!

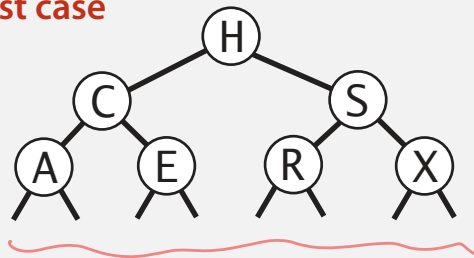
Cost. Number of compares is equal to 1 + depth of node.

Tree shape $\rightarrow$ linked list representation

- Many BSTs correspond to same set of keys.
- Number of compares for search/insert is equal to $1 + \text{depth of node}$.

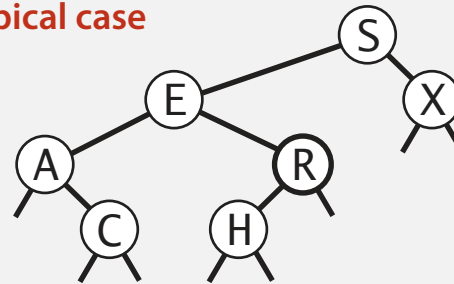
$$O(n) \rightarrow O(\log_2 n)$$

best case



Proper binary Search Tree
 $O(\log_2 n)$

typical case



$O(\log_2 n) \sim O(n)$

worst case

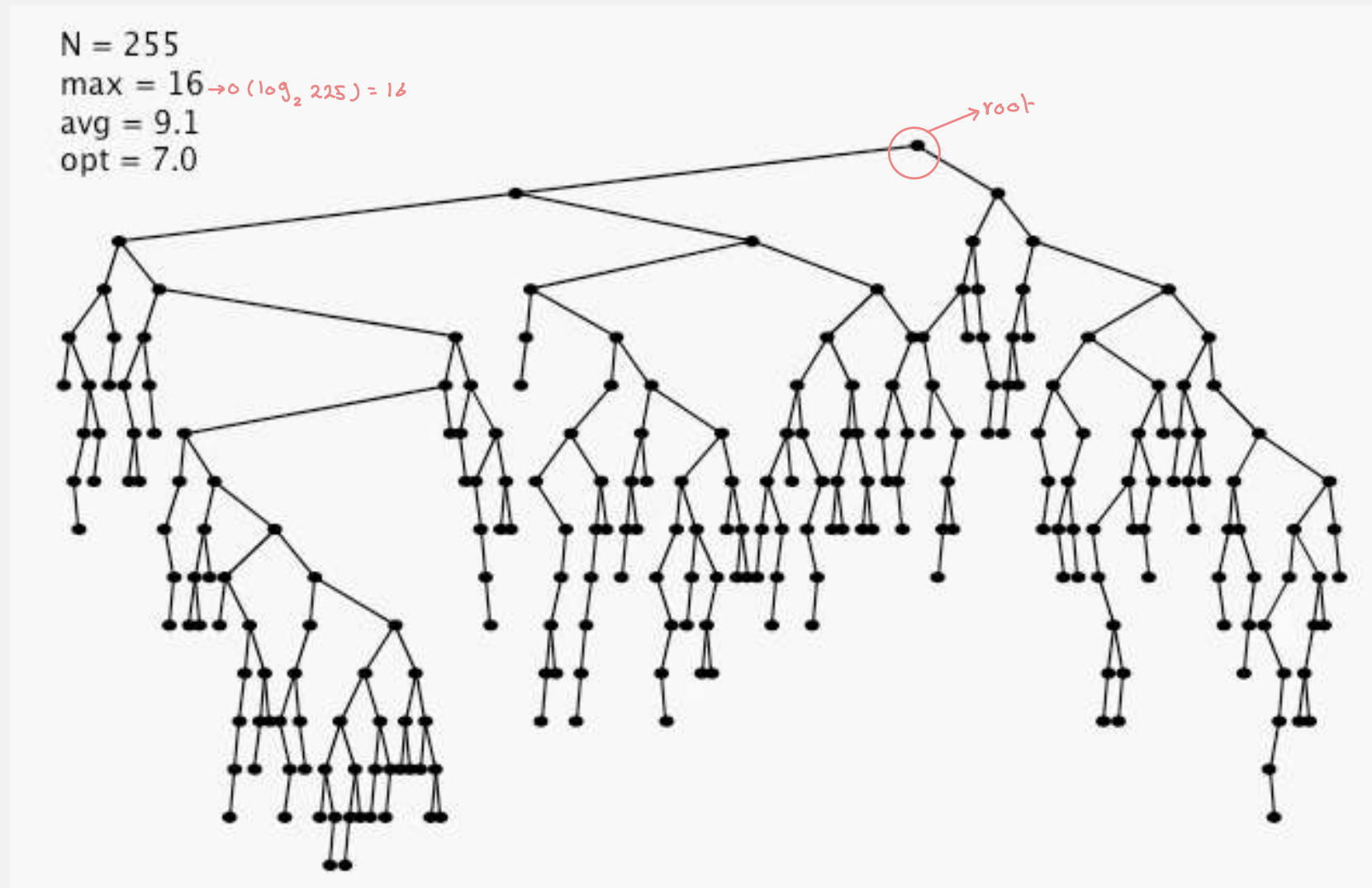


$O(n)$

Bottom line. Tree shape depends on order of insertion.

BST insertion: random order visualization

Ex. Insert keys in random order.



BSTs: mathematical analysis

Proposition. If N distinct keys are inserted into a BST in **random** order, the expected number of compares for a search/insert is $\sim 2 \ln N$.

Pf. 1–1 correspondence with quicksort partitioning.

Proposition. [Reed, 2003] If N distinct keys are inserted in random order, expected height of tree is $\sim 4.311 \ln N$.



How Tall is a Tree?

Bruce Reed
CNRS, Paris, France
reed@moka.ccr.jussieu.fr

ABSTRACT

Let H_n be the height of a random binary search tree on n nodes. We show that there exists constants $\alpha = 4.31107\dots$ and $\beta = 1.95\dots$ such that $\mathbf{E}(H_n) = \alpha \log n - \beta \log \log n + O(1)$. We also show that $\text{Var}(H_n) = O(1)$.

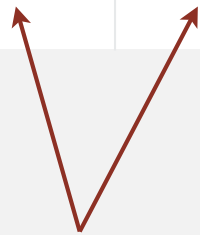
But... Worst-case height is $N - 1$.

[exponentially small chance when keys are inserted in random order]

ST implementations: summary

WCS

implementation	guarantee		average case		operations on keys
	search	insert	search hit	insert	
sequential search (unordered list)	N	N	$\frac{1}{2} N$	N	equals()
binary search (ordered array)	$\lg N$	N	$\lg N$	$\frac{1}{2} N$	compareTo()
BST	N	N	$1.39 \lg N$	$1.39 \lg N$	compareTo()



Why not shuffle to ensure a (probabilistic) guarantee of $4.311 \ln N$?



<http://algs4.cs.princeton.edu>

3.2 BINARY SEARCH TREES

- ▶ *BSTs*
- ▶ *ordered operations*
- ▶ *deletion*

ST implementations: summary

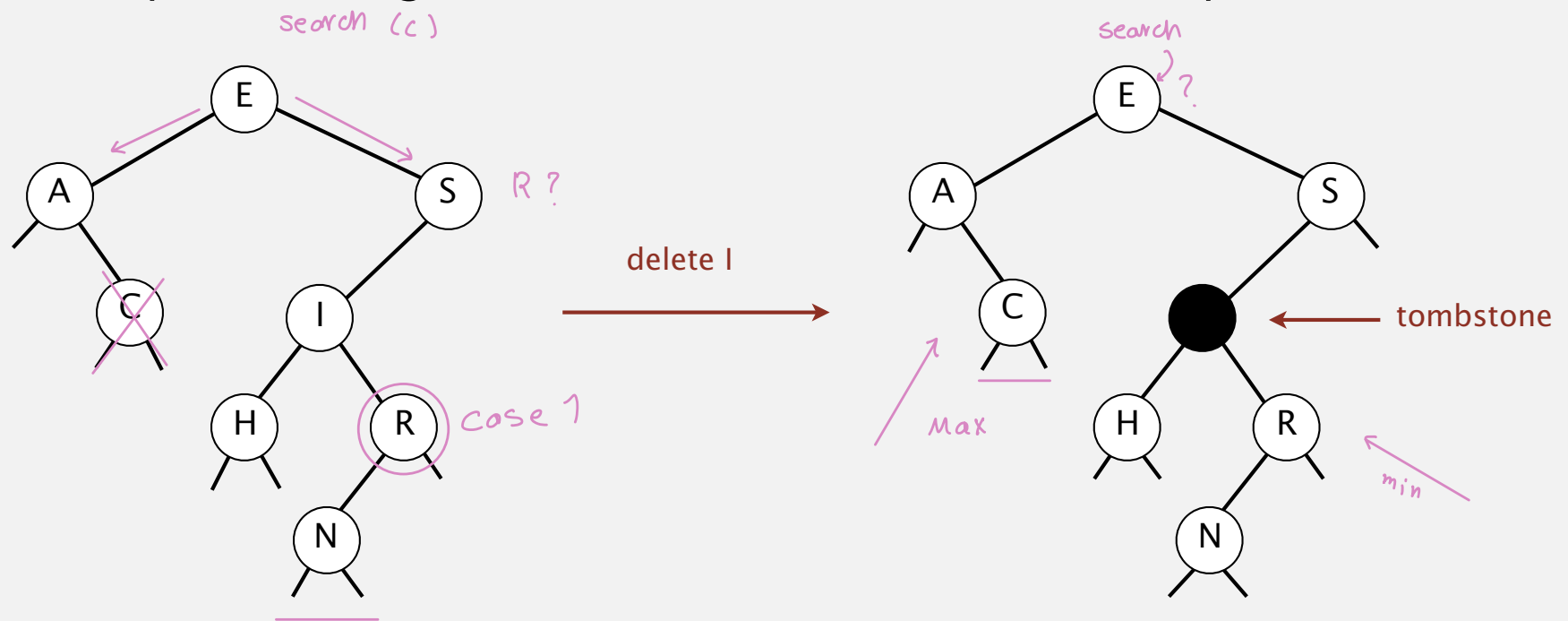
implementation	guarantee			average case			ordered ops?	operations on keys
	search	insert	delete	search hit	insert	delete		
sequential search (linked list)	N	N	N	$\frac{1}{2} N$	N	$\frac{1}{2} N$		<code>equals()</code>
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2} N$	$\frac{1}{2} N$	✓	<code>compareTo()</code>
BST	N	N	N	$1.39 \lg N$	$1.39 \lg N$	???	✓	<code>compareTo()</code>

Next. Deletion in BSTs.

BST deletion: lazy approach

To remove a node with a given key:

- Set its value to null.
- Leave key in tree to guide search (but don't consider it equal in search).



Cost. $\sim 2 \ln N'$ per insert, search, and delete (if keys in random order), where N' is the number of key-value pairs ever inserted in the BST.

Unsatisfactory solution. Tombstone (memory) overload.

Deletion

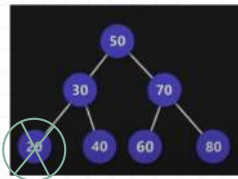
◆ Deletion in BST has 3 cases:

- Node to be deleted is a leaf node has no child
- Node to be deleted has only one child
- Node to be deleted has two children

Deletion

◆ Case 1:

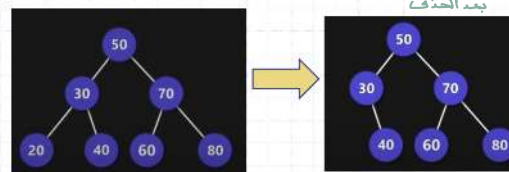
- Delete node with value '20' → اول شي نبحث عن الـ 20 .
ثاني شي اعد اولادها .
اذا ما عندها بلغي الـ node .



Deletion

◆ Case 1:

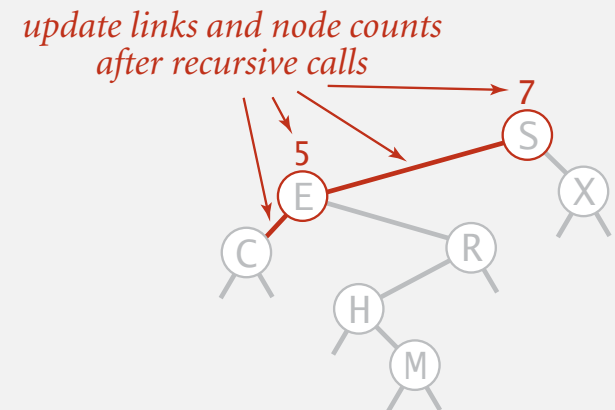
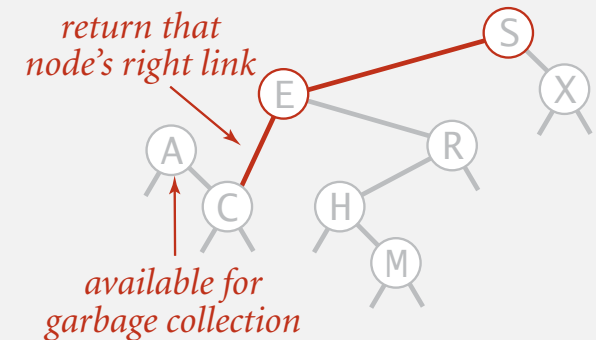
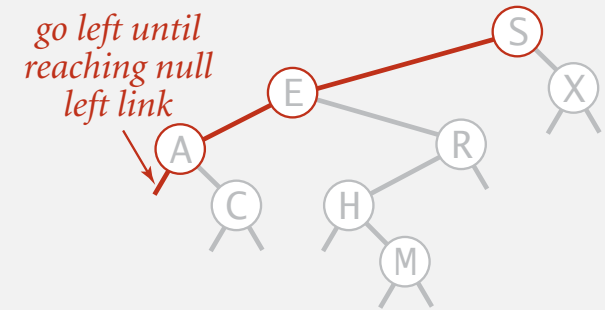
- Delete node with value '20'



Deleting the minimum

To delete the minimum key:

- Go left until finding a node with a null left link.
- Replace that node by its right link.
- Update subtree counts.



```
public void deleteMin()
{ root = deleteMin(root); }

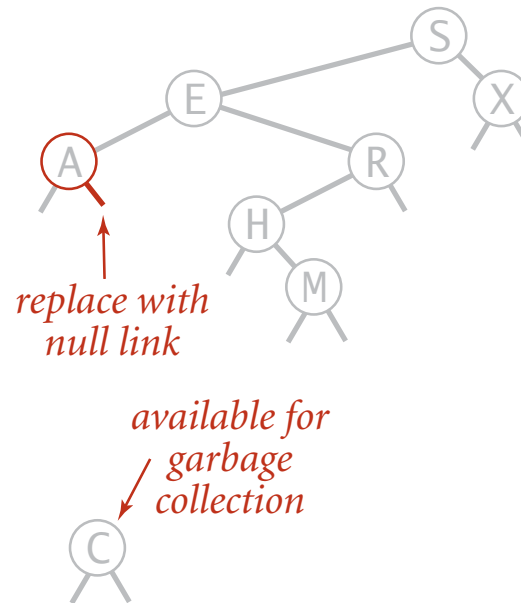
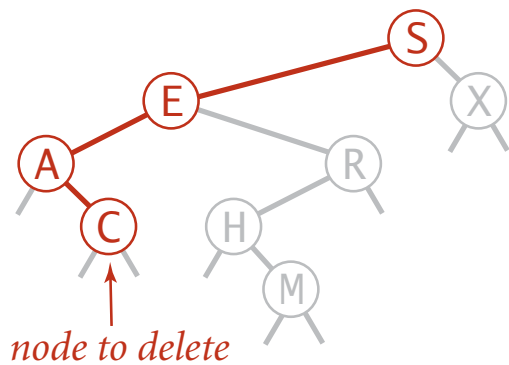
private Node deleteMin(Node x)
{
    if (x.left == null) return x.right;
    x.left = deleteMin(x.left);
    x.count = 1 + size(x.left) + size(x.right);
    return x;
}
```

Hibbard deletion *Case 2 :*

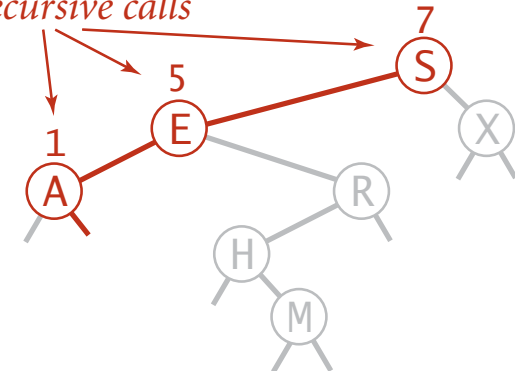
To delete a node with key k : search for node t containing key k .

Case 0. [0 children] Delete t by setting parent link to null.

deleting C



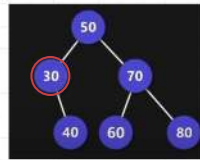
update counts after recursive calls



Deletion

◆ Case 2

- Delete node with value '30' →

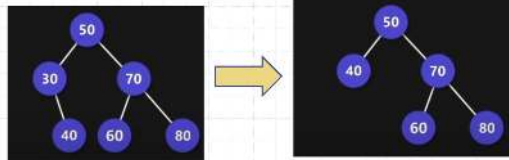


اولاً نبحث عن الـ 30
مثاني سن اعد اولاده
عنده ابن واحد
سواء يسار او يمين، بمثل الأب بالابن دائماً

Deletion

◆ Case 2

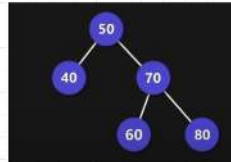
- Delete node with value '30'



Deletion

Case 3

- Delete node with value '50' →

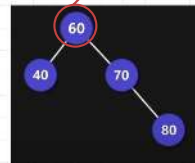
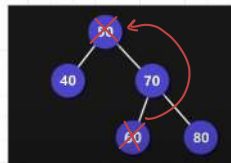


اولاً نبحث عن الـ 50.
ثانياً شيء مهم جداً الأولاد
منه Two child.
ما جبر النماذج الـ node مرفوعة.
بوزم يجب واحد يغير مكانه 50 اذا فكيت الـ node راح
تفيع الـ tree.
راح نجيب واحد مناسب يغير مكانه.
— يا باخذ اكبر node من جهة الـ left . one left, right
— او ياخذ اصغر node من جهة الـ right . one right, left

Deletion

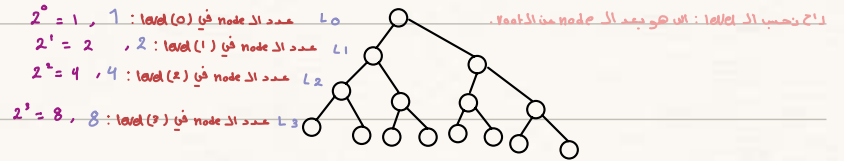
Case 3

- Delete node with value '50' →



صو اخذنا الـ 60 الي مو اصغر node
من جهة الـ right.

كم عدد الـ node في الـ level معين ؟



لا يشترط العلاقة بين الـ level وعدد الـ nodes :

BT : $O(n)$

level
 2 = number of node
 عدد النودات في level معين .

لا يشترط هو اقل عدد في النودات : node واحد

لا يشترط الـ height : E ← آخر level + 1

لا يشترط هو الـ maximum : راجع اجماع

$$2^4 - 1 = 15 \leftarrow 1 + 2 + 4 + 8 = 15$$

height : بعد ما وانتهى من الارض :

depth : من فوق انتهت .



balance : فرق الـ level من الجبهة اليمين ومن الجبهة اليسار ، لانّه يكون (1 , 0 , -1) .

left-right
 or
 right-left

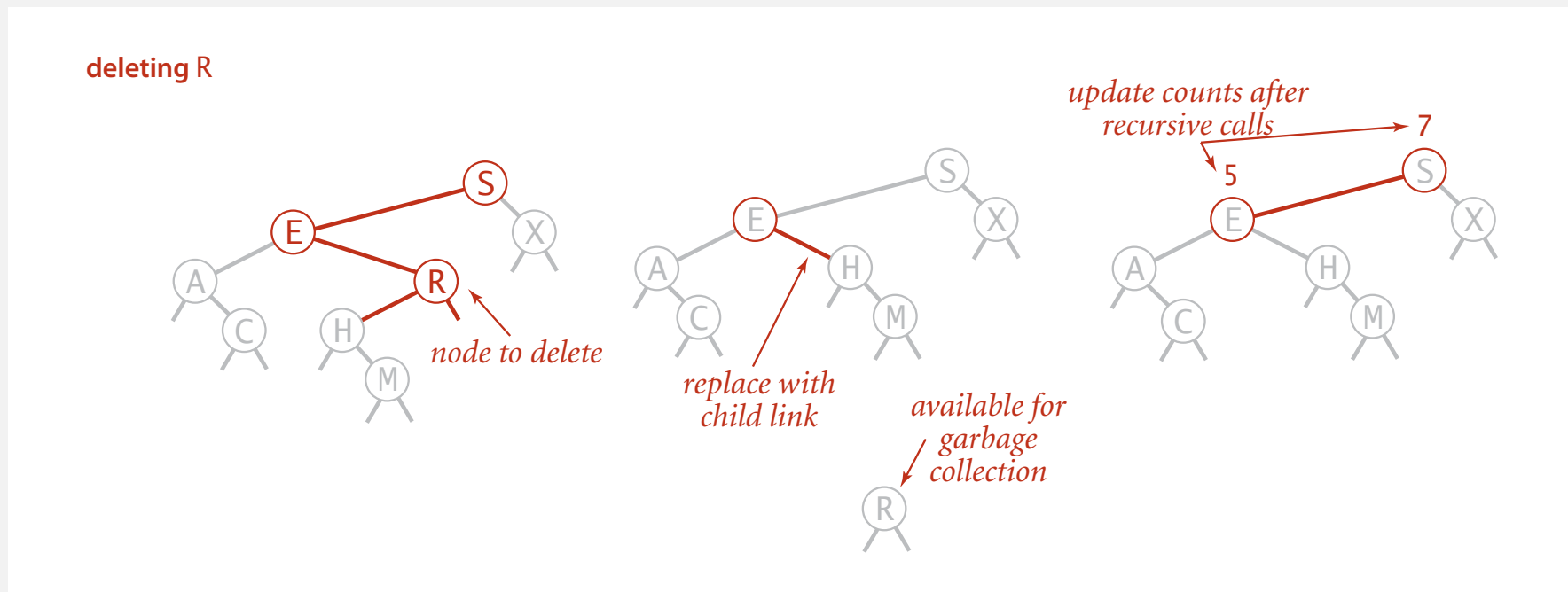
$h(F) = 0$
 $h(D) = 0$
 $h(C) = 0$
 $h(E) = 1$
 $h(B) = 2$
 $h(A) = 3$

d tree = 3 → h
 عدد edges بعد النود من
 الرؤس .

Hibbard deletion

To delete a node with key k : search for node τ containing key k .

Case 1. [1 child] Delete τ by replacing parent link.

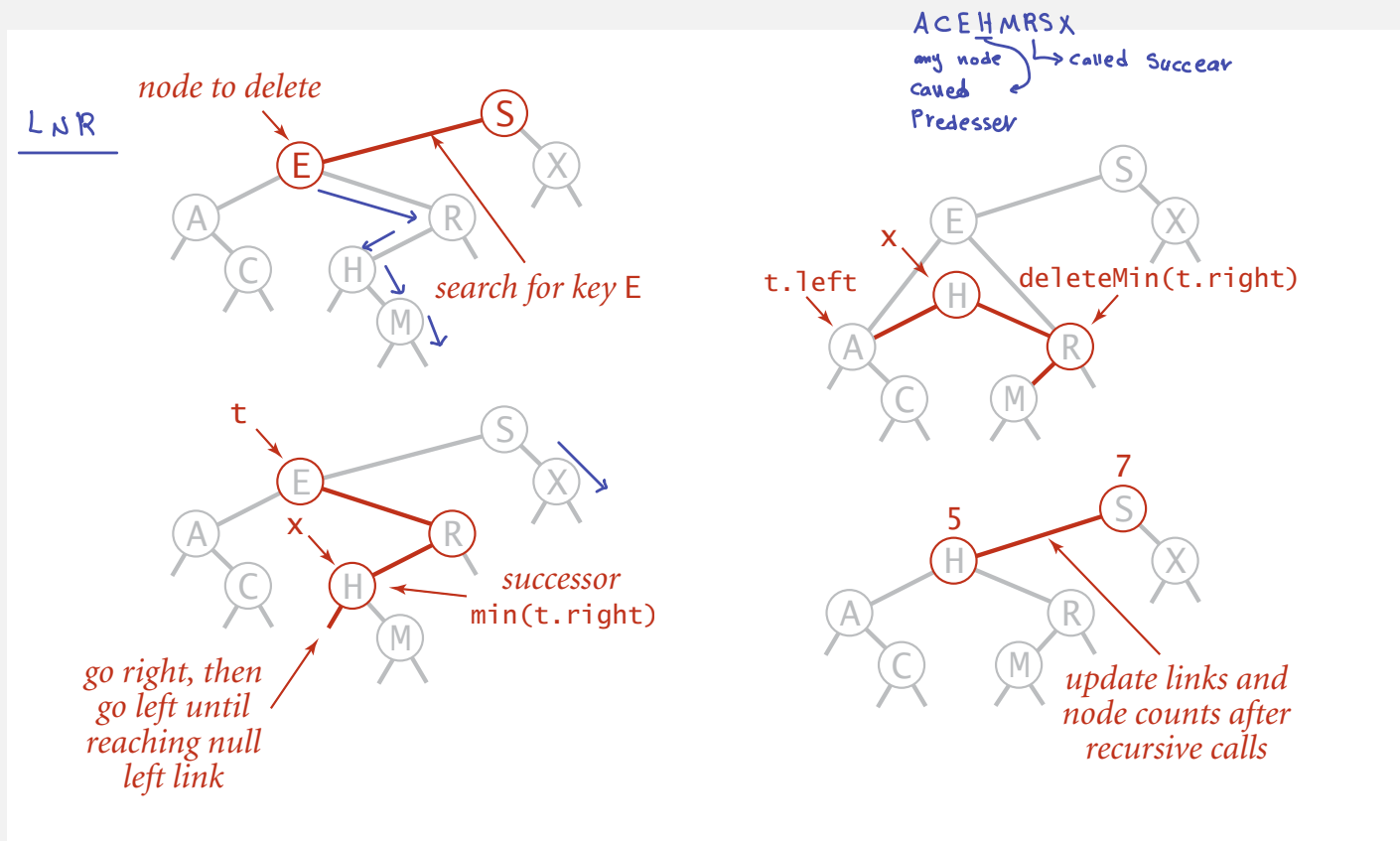


Hibbard deletion

To delete a node with key k : search for node t containing key k .

Case 2. [2 children]

- Find successor x of t .
← x has no left child
- Delete the minimum in t 's right subtree.
← but don't garbage collect x
- Put x in t 's spot.
← still a BST



Hibbard deletion: Java implementation

```
public void delete(Key key)
{ root = delete(root, key); }
```

→ + insert start from root.

```
private Node delete(Node x, Key key) {
```

```
    if (x == null) return null;
```

```
    int cmp = key.compareTo(x.key);
```

```
    if (cmp < 0) x.left = delete(x.left, key);
```

childe

search for key

```
    else if (cmp > 0) x.right = delete(x.right, key);
```

```
    else {
```

```
        if (x.right == null) return x.left;
```

childe case: (1 childe)

no right child

```
        if (x.left == null) return x.right;
```

no left child

element found

```
        Node t = x;
```

```
        x = min(t.right); x = max(t.left)
```

replace with
successor

```
        x.right = deleteMin(t.right);
```

```
        x.left = t.left;
```

```
    }
```

```
    update size after delete  
    x.count = size(x.left) + size(x.right) + 1;
```

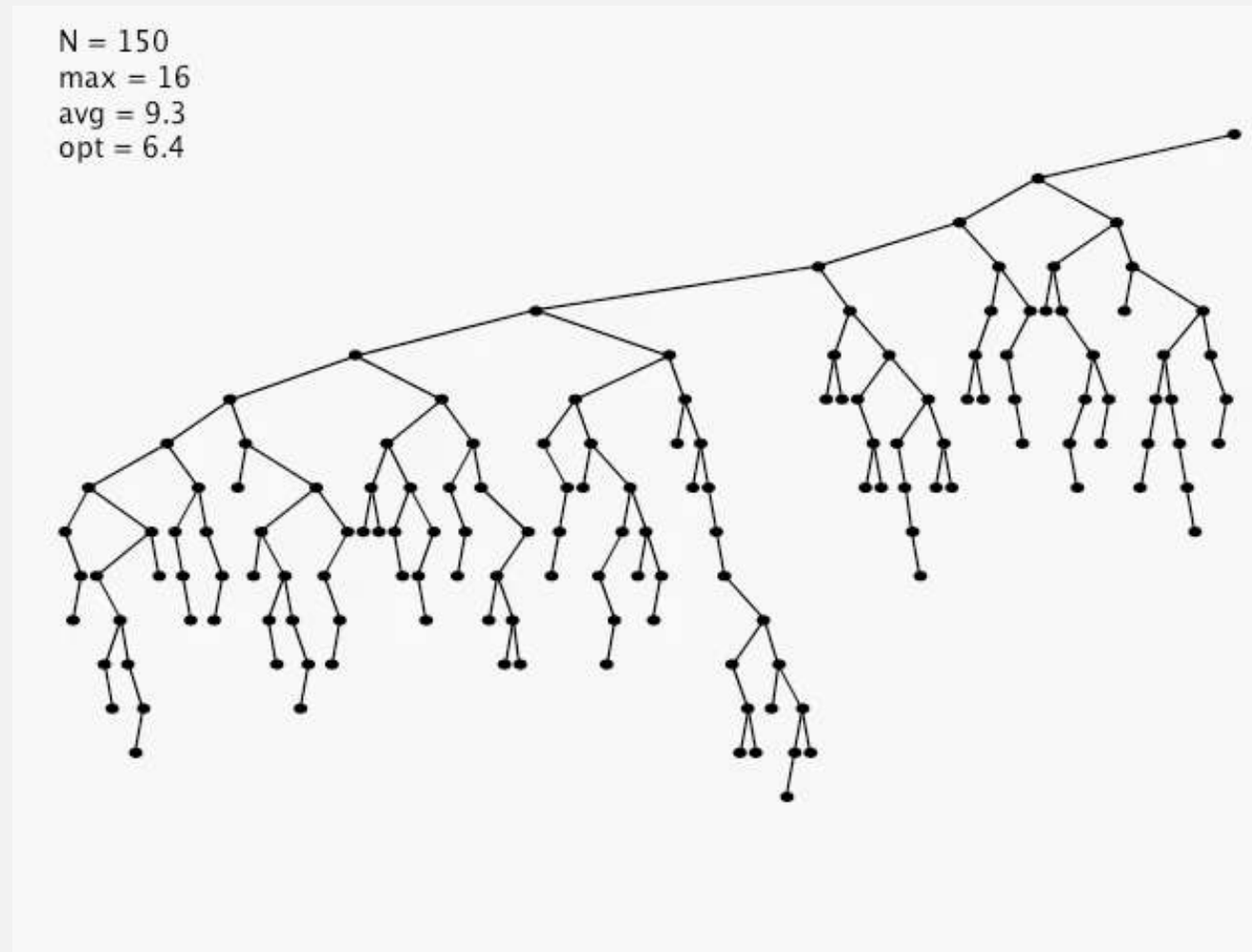
update subtree
counts

```
    return x;
```

```
}
```

Hibbard deletion: analysis

Unsatisfactory solution. Not symmetric.



Surprising consequence. Trees not random (!) $\Rightarrow \sqrt{N}$ per op.

Longstanding open problem. Simple and efficient delete for BSTs.

ST implementations: summary

W.C.S (تكون على جهة واحدة)

b.c.s (مرتبة)

$O(n)$

$\star O(\log_2 n)$

implementation	guarantee			average case			ordered ops?	operations on keys
	search	insert	delete	search hit	insert	delete		
sequential search (linked list)	N	N	N	$\frac{1}{2} N$	N	$\frac{1}{2} N$		equals()
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2} N$	$\frac{1}{2} N$	✓	compareTo()
BST	N	N	N	$1.39 \lg N$	$1.39 \lg N$	$\sqrt{N}$	✓	compareTo()

$\star$ search best case : $\log n$

(Preorder, inorder, Postorder) Travers : n

other operations also become $\sqrt{N}$
if deletions allowed

Next lecture. **Guarantee** logarithmic performance for all operations.

Method	Running Time
size, isEmpty	$O(1)$
root, parent, left, right, sibling, children, numChildren	$O(1)$
isInternal, isExternal, isRoot	$O(1)$
addRoot, addLeft, addRight, set, attach, remove	$O(1)$
depth(p)	$O(d_p + 1)$
height	$O(n)$

Table 8.1: Running times for the methods of an n -node binary tree implemented with a linked structure. The space usage is $O(n)$